want to extract them out into separate dimensions again. We only need to tell it the size of *h* or *w*, and it can infer the size of the other dimension.

The main change for multi-head attention is that when we project the initial node matrix $N:\mathbb{R}^{b\times n\times f}$ into key, query, and value matrices, we add an extra head dimension: $Q,K,V:\mathbb{R}^{b\times h\times n\times d}$, where *b* is the batch dimension, and *h* is the head dimension. We will (arbitrarily) set the number of heads to be 3 for this example, so $h = 3$, $n = 7 * 7 = 49$, $d = 64$, where *n* is the number of nodes (which is just the total number of grid positions in view), and *d* is the dimensionality of the node feature vectors, which is just something we choose empirically to be 64, but smaller or larger values might work just as well.

We will need to do a tensor contraction between the query and key tensors to get an attention tensor, $A:\mathbb{R}^{b\times h\times n\times n}$, pass it through a softmax, contract this with the value tensor, collapse the head dimension with the last *n* dimension, and contract the last (collapsed) dimension with a linear layer to get our updated node tensor, $N:\mathbb{R}^{b\times n\times d}$, which we can then pass through another self-attention layer or collapse all the nodes into a single vector and pass it through some linear layers to the final output. We will stick with a single-attention layer for all examples.

First we'll go over some specific lines in the code that are different from the single-head attention model; the full model is reproduced in listing 10.7. In order to use PyTorch's built-in linear layer module (which is just a matrix multiplication plus a bias vector), we will create a linear layer where the last dimension size is expanded by the number of attention heads.

```
>>> self.proj_shape = (self.conv4_ch+self.sp_coord_dim,self.n_heads
* self.node_size)
>>> self.k_proj = nn.Linear(*self.proj_shape)
>>> self.q_proj = nn.Linear(*self.proj_shape)
>>> self.v_proj = nn.Linear(*self.proj_shape)
```

We make three separate, ordinary linear layers just as we did for the single-head attention model, but this time we'll expand the last dimension by multiplying it by the number of attention heads. The input to these projection layers is a batch of initial node matrices, $N:\mathbb{R}^{b\times n\times c}$, and the *c*